



CS-317

Operating Systems

LECTURE 17



Concurrency

BOOK 1 CHAPTER 5



In the last lecture...

- ▶ We explored interprocess communication using message passing.
- ▶ We learned how to use messages to enforce mutual exclusion.
- ▶ We learned how to use messages to solve the producer / consumer problem.
- ▶ We explored the readers / writers problem.



Classical Synchronization Problems

THE READERS / WRITERS PROBLEM (RWP)



```

/* program readersandwriters */
int readcount, writecount;
void reader()
{
    while (true){
        semWait (z);
        semWait (rsem);
        semWait (x);
        readcount++;
        if (readcount == 1)
            semWait (wsem);
        semSignal (x);
        semSignal (rsem);
        semSignal (z);
        READUNIT();
        semWait (x);
        readcount--;
        if (readcount == 0) semSignal (wsem);
        semSignal (x);
    }
}
void writer ()
{
    while (true){
        semWait (y);
        writecount++;
        if (writecount == 1)
            semWait (rsem);
        semSignal (y);
        semWait (wsem);
        WRITEUNIT();
        semSignal (wsem);
        semWait (y);
        writecount--;
        if (writecount == 0) semSignal (rsem);
        semSignal (y);
    }
}
void main()
{
    readcount = writecount = 0;
    parbegin (reader, writer);
}

```

prev code had reader priority

Errors in code:

1. The semaphores have not been initialized. All of them must be globally defined and initialized to 1.
2. The third last line of code in the writer should be **writecount--;**

Figure 5.23 A Solution to the Readers/Writers Problem Using Semaphore: Writers Have Priority



```

/* program readersandwriters */
int readcount, writecount;
void reader()
{
    while (true){
        semWait (z);
        semWait (rsem);
        semWait (x);
        readcount++;
        if (readcount == 1)
            semWait (wsem);
        semSignal (x);
        semSignal (rsem);
        semSignal (z);
        READUNIT();
        semWait (x);
        readcount--;
        if (readcount == 0) semSignal (wsem);
        semSignal (x);
    }
}

void writer ()
{
    while (true){
        semWait (y);
        writecount++;
        if (writecount == 1)
            semWait (rsem);
        semSignal (y);
        semWait (wsem);
        WRITEUNIT();
        semSignal (wsem);
        semWait (y);
        writecount--;
        if (writecount == 0) semSignal (rsem);
        semSignal (y);
    }
}

void main()
{
    readcount = writecount = 0;
    parbegin (reader, writer);
}

```

Figure 5.23 A Solution to the Readers/Writers Problem Using Semaphore: Writers Have Priority

For writers, the following semaphores and variables are added to the ones already defined:

1. A semaphore **rsem** that inhibits all readers while there is at least one writer desiring access to the data area.
2. A variable **writecount** that controls the setting of **rsem**.
3. A semaphore **y** that controls the updating of **writecount**.




```

/* program readersandwriters */
int readcount, writecount;
void reader()
{
    while (true){
        semWait (z); nested lock
        semWait (rsem);
        semWait (x);
        readcount++;
        if (readcount == 1)
            semWait (wsem);
        semSignal (x);
        semSignal (rsem);
        semSignal (z);
        READUNIT();
        semWait (x);
        readcount--;
        if (readcount == 0) semSignal (wsem);
        semSignal (x);
    }
}

void writer ()
{
    while (true){
        semWait (y);
        writecount++;
        if (writecount == 1)
            semWait (rsem); first writer has arrived block all readers,
        semSignal (y); writecount lock released
        semWait (wsem); lock acquired on writing
        WRITEUNIT();
        semSignal (wsem);
        semWait (y);
        writecount--;
        if (writecount == 0) semSignal (rsem);
        semSignal (y);
    }
}

void main()
{
    readcount = writecount = 0;
    parbegin (reader, writer);
}

```

readcount tells how many readig not including the blocked readers since line 9 R1v blocked before incrementing readcount

only first writer blocks because the next writer comming the readers already blocked

first writer has arrived block all readers, writecount lock released lock acquired on writing



For readers, one additional semaphore is needed. A long queue must not be allowed to build up on **rsem**; otherwise writers will not be able to jump the queue. Therefore, only one reader is allowed to queue on **rsem**, with any additional readers queueing on semaphore **z**, immediately before waiting on **rsem**.

Figure 5.23 A Solution to the Readers/Writers Problem Using Semaphore: Writers Have Priority



```

/* program readersandwriters */
int readcount, writecount;
void reader()
{
    while (true){
        semWait (z);
        semWait (rsem);
        semWait (x);
        readcount++;
        if (readcount == 1)
            semWait (wsem);
        semSignal (x);
        semSignal (rsem);
        semSignal (z);
        READUNIT();
        semWait (x);
        readcount--;
        if (readcount == 0) semSignal (wsem);
        semSignal (x);
    }
}
void writer ()
{
    while (true){
        semWait (y);
        writecount++;
        if (writecount == 1)
            semWait (rsem);
        semSignal (y);
        semWait (wsem);
        WRITEUNIT();
        semSignal (wsem);
        semWait (y);
        writecount--;
        if (writecount == 0) semSignal (rsem);
        semSignal (y);
    }
}
void main()
{
    readcount = writecount = 0;
    parbegin (reader, writer);
}

```

	Process	Action	Variables
1	All semaphores are 1, readcount=writecount=0		
2	Writer 1	semWait(y)	y=0(empty)
3		writecount++	writecount=1
4		semWait(rsem)	rsem=0(empty)
5		semSignal(y)	y=1(empty)
6		semWait(wsem)	wsem=0(empty)
7	writing will take time hence another process catered	WRITEUNIT	
8	Reader 1	semWait(z)	z=0(empty)
9	R1 blocks	semWait(rsem)	rsem=-1(R1)
10	Reader 2 blocks	semWait(z)	z=-1(R2)
11	Writer 2	semWait(y)	y=0(empty)
12		writecount++	writecount=2
13		semSignal(y)	y=1(empty)
14	W2 blocks	semWait(wsem)	wsem=-1(W2)
15	Reader 3 blocks	semWait(z)	z=-2(R2, R3)
16	W1 (W2 unblocks)	semSignal(wsem)	wsem=0(empty)

Figure 5.23 A Solution to the Readers/Writers Problem Priority


```

/* program readersandwriters */
int readcount, writecount;
void reader()
{
    while (true){
        semWait (z);
        semWait (rsem);
        semWait (x);
        readcount++;
        if (readcount == 1)
            semWait (wsem);
        semSignal (x);
        semSignal (rsem);
        semSignal (z);
        READUNIT();
        semWait (x);
        readcount--;
        if (readcount == 0) semSignal (wsem);
        semSignal (x);
    }
}
void writer ()
{
    while (true){
        semWait (y);
        writecount++;
        if (writecount == 1)
            semWait (rsem);
        semSignal (y);
        semWait (wsem);
        WRITEUNIT();
        semSignal (wsem);
        semWait (y);
        writecount--;
        if (writecount == 0) semSignal (rsem);
        semSignal (y);
    }
}
void main()
{
    readcount = writecount = 0;
    parbegin (reader, writer);
}

```

	Process	Action	Variables
	x=1 (empty) y=1 (empty) z=-2(R2, R3) rsem=-1(R1) wsem=0(empty) readcount=0 writecount=2		
17	Writer 2	WRITEUNIT	
18	Writer 1	semWait(y)	y=0(empty)
19		writecount--	writecount=1
20	W1 leaves	semSignal(y)	y=1 (empty)
21	Writer 2	semSignal(wsem)	wsem=1 (empty)
22		semWait(y)	y=0(empty)
23		writecount--	writecount=0
24	W2 (R1 unblocks)	semSignal(rsem)	rsem=0(empty)
25	W2 leaves	semSignal(y)	y=1 (empty)
26	Reader 1	semWait(x)	x=0(empty)
27		readcount++	readcount=1
28		semWait(wsem)	wsem=0(empty)
29		semSignal(x)	x=1 (empty)
30		semSignal(rsem)	rsem=1 (empty)

Figure 5.23 A Solution to the Readers/Writers Problem Priority

```

/* program readersandwriters */
int readcount, writecount;
void reader()
{
    while (true){
        semWait (z);
        semWait (rsem);
        semWait (x);
        readcount++;
        if (readcount == 1)
            semWait (wsem);
        semSignal (x);
        semSignal (rsem);
        semSignal (z);
        READUNIT();
        semWait (x);
        readcount--;
        if (readcount == 0) semSignal (wsem);
        semSignal (x);
    }
}
void writer ()
{
    while (true){
        semWait (y);
        writecount++;
        if (writecount == 1)
            semWait (rsem);
        semSignal (y);
        semWait (wsem);
        WRITEUNIT();
        semSignal (wsem);
        semWait (y);
        writecount--;
        if (writecount == 0) semSignal (rsem);
        semSignal (y);
    }
}
void main()
{
    readcount = writecount = 0;
    parbegin (reader, writer);
}

```

fifo

readingtaskstime



	Process	Action	Variables
	x=1 (empty) y=1 (empty) z=-2(R2, R3) rsem=1 (empty) wsem=0 (empty) readcount=1 writecount=0		
31	R1 (R2 unblocks)	semSignal(z)	z=-1 (R3)
32	Reader 1	READUNIT	
33	Writer 3	semWait(y)	y=0 (empty)
34		writecount++	writecount=1
35		semWait(rsem)	rsem=0 (empty)
36		semSignal(y)	y=1 (empty)
37	W3 blocks	semWait(wsem)	wsem=-1 (W3)
38	Reader 2 blocks	semWait(rsem)	rsem=-1 (R2)
39	Reader 1	semWait(x)	x=0 (empty)
40		readcount--	readcount=0
41	R1 (W3 unblocks)	semSignal(wsem)	wsem=0 (empty)
42	Reader 1 leaves	semSignal(x)	x=1 (empty)
43	Writer 3	WRITEUNIT	
44		semSignal(wsem)	wsem=1 (empty)

Figure 5.23 A Solution to the Readers/Writers Problem Priority

```

/* program readersandwriters */
int readcount, writecount;
void reader()
{
    while (true){
        semWait (z);
        semWait (rsem);
        semWait (x);
        readcount++;
        if (readcount == 1)
            semWait (wsem);
        semSignal (x);
        semSignal (rsem);
        semSignal (z);
        READUNIT();
        semWait (x);
        readcount--;
        if (readcount == 0) semSignal (wsem);
        semSignal (x);
    }
}
void writer ()
{
    while (true){
        semWait (y);
        writecount++;
        if (writecount == 1)
            semWait (rsem);
        semSignal (y);
        semWait (wsem);
        WRITEUNIT();
        semSignal (wsem);
        semWait (y);
        writecount--;
        if (writecount == 0) semSignal (rsem);
        semSignal (y);
    }
}
void main()
{
    readcount = writecount = 0;
    parbegin (reader, writer);
}

```

	Process	Action	Variables
	x=1 (empty) y=1 (empty) z=-1 (R3) rsem=-1 (R2) wsem=1 (empty) readcount=0 writecount=1		
45	Writer 3	semWait(y)	y=0(empty)
46		writecount--	writecount=0
47	W3 (R2 unblocks)	semSignal(rsem)	rsem=0(empty)
48	W3 leaves	semSignal(y)	y=1 (empty)
49	Reader 2	semWait(x)	x=0(empty)
50		readcount++	readcount=1
51		semWait(wsem)	wsem=0(empty)
52		semSignal(x)	x=1 (empty)
53		semSignal(rsem)	rsem=1 (empty)
54	R2 (R3 unblocks)	semSignal(z)	z=0(empty)
55	Reader 2	READUNIT	
56	Reader 3	semWait(rsem)	rsem=0(empty)
57		semWait(x)	x=0(empty)
58		readcount++	readcount=2

Figure 5.23 A Solution to the Readers/Writers Problem Priority


```

/* program readersandwriters */
int readcount, writecount;
void reader()
{
    while (true){
        semWait (z);
        semWait (rsem);
        semWait (x);
        readcount++;
        if (readcount == 1)
            semWait (wsem);
        semSignal (x);
        semSignal (rsem);
        semSignal (z);
        READUNIT();
        semWait (x);
        readcount--;
        if (readcount == 0) semSignal (wsem);
        semSignal (x);
    }
}
void writer ()
{
    while (true){
        semWait (y);
        writecount++;
        if (writecount == 1)
            semWait (rsem);
        semSignal (y);
        semWait (wsem);
        WRITEUNIT();
        semSignal (wsem);
        semWait (y);
        writecount--;
        if (writecount == 0) semSignal (rsem);
        semSignal (y);
    }
}
void main()
{
    readcount = writecount = 0;
    parbegin (reader, writer);
}

```

	Process	Action	Variables
	x=0(empty) y=1(empty) z=0(empty) rsem=0(empty) wsem=0(empty) readcount=2 writecount=0		
59	Reader 3	semSignal(x)	x=1 (empty)
60		semSignal(rsem)	rsem=1 (empty)
61		semSignal(z)	z=1 (empty)
62		READUNIT	
63		semWait(x)	x=0(empty)
64		readcount--	readcount=1
65	R3 leaves	semSignal(x)	x=1 (empty)
66	Reader 2	semWait(x)	x=0(empty)
67		readcount--	readcount=0
68		semSignal(wsem)	wsem=1 (empty)
69	R2 leaves	semSignal(x)	x=1 (empty)
	x=1 (empty) y=1 (empty) z=1 (empty) rsem=1 (empty) wsem=1 (empty) readcount=0 writecount=0		

Figure 5.23 A Solution to the Readers/Writers Problem Using Semaphore: Writers Have Priority



Table 5.6 State of the Process Queues for Program of Figure 5.23

Readers only in the system	• <i>wsem</i> set if(readcount == 1) semWait(wsem); this allows no writer to enter • no queues
Writers only in the system	• <i>wsem</i> and <i>rsem</i> set readers writers all blocked since lock applied • writers queue on <i>wsem</i>
Both readers and writers with read first	• <i>wsem</i> set by reader • <i>rsem</i> set by writer • all writers queue on <i>wsem</i> • one reader queues on <i>rsem</i> • other readers queue on <i>z</i>
Both readers and writers with write first	• <i>wsem</i> set by writer • <i>rsem</i> set by writer • writers queue on <i>wsem</i> • one reader queues on <i>rsem</i> • other readers queue on <i>z</i>



```
void reader (int i) {
message rmsg;
while (true) {
    rmsg = i; id of process sent as msg
    send (readrequest, rmsg); readrequest mailbox
    receive (rbox [i], rmsg); rbox is another mailbox
    READUNIT ();
    rmsg = i;
    send (finished, rmsg); }
}
```

```
void writer (int j) {
message wmsg;
while (true) {
    wmsg = j; id stored in wmsg
    send (writerequest, wmsg);
    receive (wbox [j], wmsg); rec block
    WRITEUNIT ();
    wmsg = j;
    send (finished, wmsg); }
}
```

blocking receive *rbox: readers' mailbox*
finished, writerequest, readrequest : controller mailbox




```
void controller () {
while (true) {
    if (count > 0) {
        if (!empty (finished) {
            receive (finished, msg);
            count++; }
        else if (!empty (writerequest) {
            receive (writerequest, msg);
            writer_id = msg;
            count = count - 100; }
        else if (!empty (readrequest) {
            receive (readrequest, msg);
            reader_id = msg;
            send (rbox [reader_id], "OK");
            count--; } }
    if (count == 0) {
        send (wbox [writer_id], "OK");
        receive (finished, msg); this will wait till writer finishes its work
        count = 100; }
    while (count < 0) {
        receive (finished, msg);
        count++; } until writers writing
    } }
}
```

A solution to readers / writers problem using message passing *writer priority*

	Process	Process' mailbox	count	finished	writerequest	readrequest
		empty	100	empty	empty	empty
1	Writer 1				[1]	
2		waiting	writer1 blocks to receive a message			
3	Controller		0		received	
4		send (wbox[1], OK)			empty	
5				waiting	block to rec a finished mail	
6	Writer 1	receives and starts writing				
7	Reader 1					[1]
8		waiting	receiving block since W1 havent send the finishd mail			
9	Writer 2				[2]	
10		waiting	feceiving block since W1 havent send the finishd mail			
11	Reader 2					[1], [2]
12		waiting	feceiving block since W1 havent send the finishd mail			
13	Writer 1 leaves			[1]	finished mail rece, controller starts executing next line of code	
14	Controller		100	receives		

	Process	Process' mailbox	count	finished	writerequest	readrequest
		R1, R2, W2 waiting	100	empty	[2]	[1], [2]
15	Controller	Writer priority hence W2 will be served	0		receives	
16		send (wbox[2], OK)			empty	
17				waiting		
18	Writer 2	receives and starts writing				
19	Writer 2 leaves			[2]		
20	Controller		100	receives		
21				empty		receives [1]
22		send (rbox[1], OK)	99			[2]
23	Reader 1	receives and starts reading				
24	Controller					receives
25		send (rbox[2], OK)	98			empty
26	Reader 2	receives and starts reading				
27	Writer 3				[3]	
28		waiting				

	Process	Process' mailbox	count	finished	writerequest	readrequest
		R1, R2 reading, W3 waiting	98	empty	[3]	empty
29	Controller	98-100 = -2 since served the write request	-2		receives	
30				waiting	empty	
31	Reader 2 leaves			[2]	Wr will keep waiting until the readers leave and count become 0	
32	Controller		-1	receives		
33				waiting		
34	Reader 1 leaves			[1]		
35	Controller		0	receives		
36		send (wbox[3], OK)		empty		
37				waiting		
38	Writer 3	receives and starts writing				
39	Writer 3 leaves			[3]		
40	Controller		100	receives		
			100	empty	empty	empty

Analysis of the message passing solution to RWP

- ▶ The technique is simple and flexible in many ways.
- ▶ Any number of readers can be accommodated. The variable `count` can be set to the maximum number of readers plus one.
- ▶ If the sequence in which the controller checks mailboxes is changed in the body of `if (count > 0)`, the same code can be made to prioritize readers. When the controller checks `writerequest` first, writers are prioritized. When it checks `readrequest` first, readers are prioritized.
- ▶ When we want to enforce ME, we make the controller send an OK message to a writer, and then we make it issue a blocking receive for a finished message from the writer. The blocking receive blocks the controller, and nobody else can be issued an OK message.



Homework:

Draw up parallel execution traces using the following conditions:

Make assumptions that you deem necessary as you go along.

1. Use the semaphore solution to RWP, writers have priority, with 3 writers and 2 readers. The order of arrival of the processes are R1, W1, W2, R2, W3.
2. Use the message passing solution to RWP, writers have priority, with 3 writers and 2 readers. The order of arrival of the processes are R1, W1, W2, R2, W3.



Lessons learned

- ▶ We explored the readers / writers problem and its various solutions.
- ▶ We realized that both the semaphore approach and the message passing approach are equally effective in solving classical synchronization problems.
- ▶ We learned how to do dry runs of long parallelly executing processes.



Things to do



Read the book.



Write your questions on a piece of paper and keep it in front of you during the online session.



If you have suggestions relevant to content or quality of this lecture, email them to me.

